



CRYPTOGRAPHY & PUBLIC KEY INFRASTRUCTURE

QUICK REFERENCE & SECURITY OPERATIONS CHEAT SHEET

1. COMMON USES OF ASYMMETRIC ENCRYPTION

Asymmetric encryption relies on a mathematically linked key pair: a **PUBLIC KEY** (encryption/verification) and a **PRIVATE KEY** (decryption/signing).

USE CASE	OPERATION MECHANISM	SECURITY GOAL
Confidentiality	Sender encrypts with receiver's Public Key . Only receiver decrypts with Private Key .	Ensures message privacy over insecure networks.
Digital Signatures	Sender hashes document & encrypts hash with Private Key . Receiver verifies with Public Key .	Guarantees Authenticity & Integrity (non-repudiation).
Key Exchange	Securely transmits a symmetric key using asymmetric keys, then switches to fast symmetric encryption.	Combines asymmetric security with symmetric speed.

2. DIFFIE-HELLMAN KEY EXCHANGE (DH)

Protocol allowing two parties (Alice & Bob) to establish a shared secret over an insecure channel without transmitting the secret itself.

STEP	ALICE	PUBLIC CHANNEL	BOB
1. Setup	Agree on parameters	Prime p , Generator g	Agree on parameters
2. Private Pick	Choose secret exp a	—	Choose secret exp b
3. Public Compute	Compute $A = g^a \text{ mod } p$	Send A → / ← B	Compute $B = g^b \text{ mod } p$
4. Shared Secret	$K = B^a \text{ mod } p$	—	$K = A^b \text{ mod } p$

Core Identity: $K = (g^b)^a \text{ mod } p = (g^a)^b \text{ mod } p = g^{ab} \text{ mod } p$

3. RSA (RIVEST-SHAMIR-ADLEMAN)

Asymmetric system based on the computational difficulty of factoring large composite prime numbers (**Prime Factorization Problem**).

Key Generation Math:

- Select large primes **p** and **q**.
- Modulus: $n = p \times q$
- Euler's Totient: $\phi(n) = (p-1)(q-1)$
- Choose **e**: $1 < e < \phi(n)$ & $\text{gcd}(e, \phi(n)) = 1$ (default: 65537)
- Private Exponent **d**:

$$d \cdot e \equiv 1 \text{ mod } \phi(n) \Rightarrow d = e^{-1} \text{ mod } \phi(n)$$

Core Formulas:

Public Key: (e, n) | **Private Key:** (d, n)

Encryption: $C = M^e \text{ mod } n$

Decryption: $M = C^d \text{ mod } n$

Signing: $S = H(M)^d \text{ mod } n$

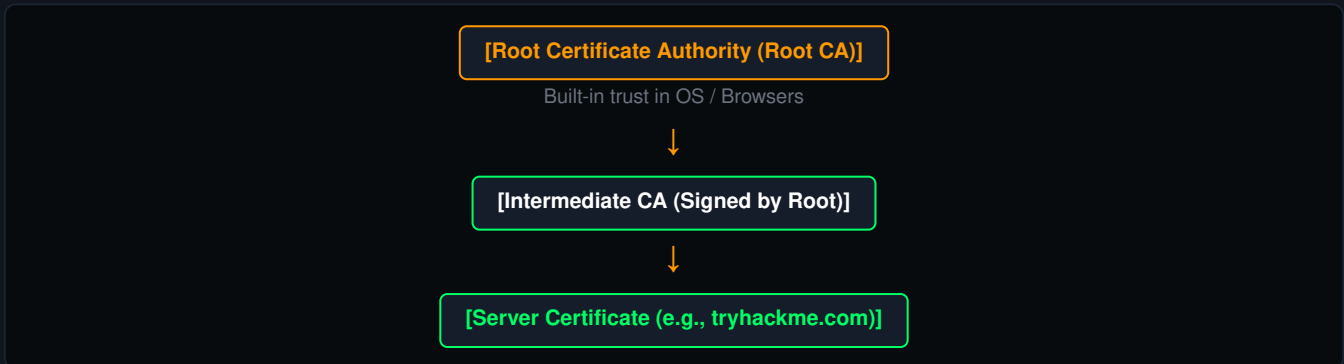
Verification: $H(M) \stackrel{?}{=} S^e \text{ mod } n$

4. DIGITAL SIGNATURES VS. ELECTRONIC SIGNATURES

FEATURE	DIGITAL SIGNATURE	ELECTRONIC SIGNATURE
Mechanism	Encrypts document hash using a Private Key	Visual image, scan, or checkbox acknowledgment
Integrity	Cryptographically guaranteed (altering 1 bit fails hash)	Weak/None (image can be pasted onto altered docs)
Authenticity	Mathematically proven via Key Pairs	Easily forged
Legal Value	Binding equivalent to physical signatures/seals	Limited legal weight depending on context

5. CERTIFICATES & CHAIN OF TRUST (PKI)

Public Key Infrastructure binds a public key to an identity (domain/org) via SSL/TLS certificates.



- **Root CAs:** Pre-installed trusted entities embedded in operating systems and web browsers.
- **Chain of Trust:** Verification cascades down: Browser trusts Root CA → Intermediate CA → Server Certificate.
- **Automated TLS:** Free, automated certificates can be obtained via **Let's Encrypt**.

6. GPG / PGP (PRETTY GOOD PRIVACY)

Open-source OpenPGP implementation (GnuPG/GPG) for encrypting, decrypting, and signing files/emails.

GPG CLI COMMAND REFERENCE

```
# Generate a new key pair interactively
gpg --full-gen-key

# Export Public Key (ASCII armor format)
gpg --export -a "User Name" > public.key

# Export Private Key (KEEP SECURE!)
gpg --export-secret-keys -a "User Name" > private.key

# Import Key (backup or partner key)
gpg --import key.key

# Encrypt file for recipient | Decrypt encrypted file | Sign file
gpg --recipient "user@domain.com" --encrypt file.txt
gpg --decrypt message.gpg
gpg --sign file.txt
```

CRACKING GPG PASSPHRASE (CTF WORKFLOW)

```
# 1. Extract hash from passphrase-protected private key
gpg2john private.key > gpg_hash.txt

# 2. Crack hash using John the Ripper
john --wordlist=/usr/share/wordlists/rockyou.txt gpg_hash.txt
```

7. SSH AUTHENTICATION & KEY MANAGEMENT

Uses asymmetric cryptography for **Server Authentication** (prevents MITM via *known_hosts*) and **Client Authentication** (replaces passwords with key pairs).

KEY ALGORITHMS

- **Ed25519**: Recommended modern standard (high performance, short keys, Curve25519).
- **RSA**: Traditional algorithm (minimum key length should be ≥ 3072 bits).
- **ECDSA / DSA**: Legacy or elliptic-curve signature algorithms.
- **ed25519-sk / ecdsa-sk**: Hardware security key integrations (e.g., YubiKey).

SSH COMMAND REFERENCE

```
# 1. Generate SSH Key Pair (Ed25519)
ssh-keygen -t ed25519 -C "user@email.com"

# 2. Copy Public Key to Remote Server automatically
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@remote_host

# 3. Connect using Private Key
ssh -i ~/.ssh/id_ed25519 user@remote_host
```

FILE SYSTEM STRUCTURE & PERMISSIONS

FILE / DIRECTORY	LOCATION	PURPOSE	REQUIRED PERMISSIONS
<i>~/.ssh</i>	Client & Server	Core SSH configurations directory	700 (drwx-----)
<i>id_ed25519</i>	Client	Private Key (NEVER SHARE)	600 (-rw-----)
<i>id_ed25519.pub</i>	Client / Server	Public Key	644 (-rw-r--r--)
<i>authorized_keys</i>	Server	Stores public keys allowed to log in	600 (-rw-----)
<i>known_hosts</i>	Client	Stores fingerprints of verified remote hosts	644 (-rw-r--r--)

MANUAL INJECTION VIA TERMINAL / REVERSE SHELL (CTFS)

```
# Create directory and configure permissions
mkdir -p ~/.ssh && chmod 700 ~/.ssh

# Append your public key to authorized_keys
echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... user@host" >> ~/.ssh/authorized_keys

# Enforce strict file permissions
chmod 600 ~/.ssh/authorized_keys
```

CRACKING SSH KEY PASSPHRASE

```
# 1. Extract hash from encrypted SSH private key
python3 /usr/share/john/ssh2john.py id_rsa > ssh_hash.txt

# 2. Crack passphrase using John the Ripper
john --wordlist=/usr/share/wordlists/rockyou.txt ssh_hash.txt

# 3. View cracked passphrase
john --show ssh_hash.txt
```